DATA226_Intermediate_Progress_Evidence.docx

Group Members :

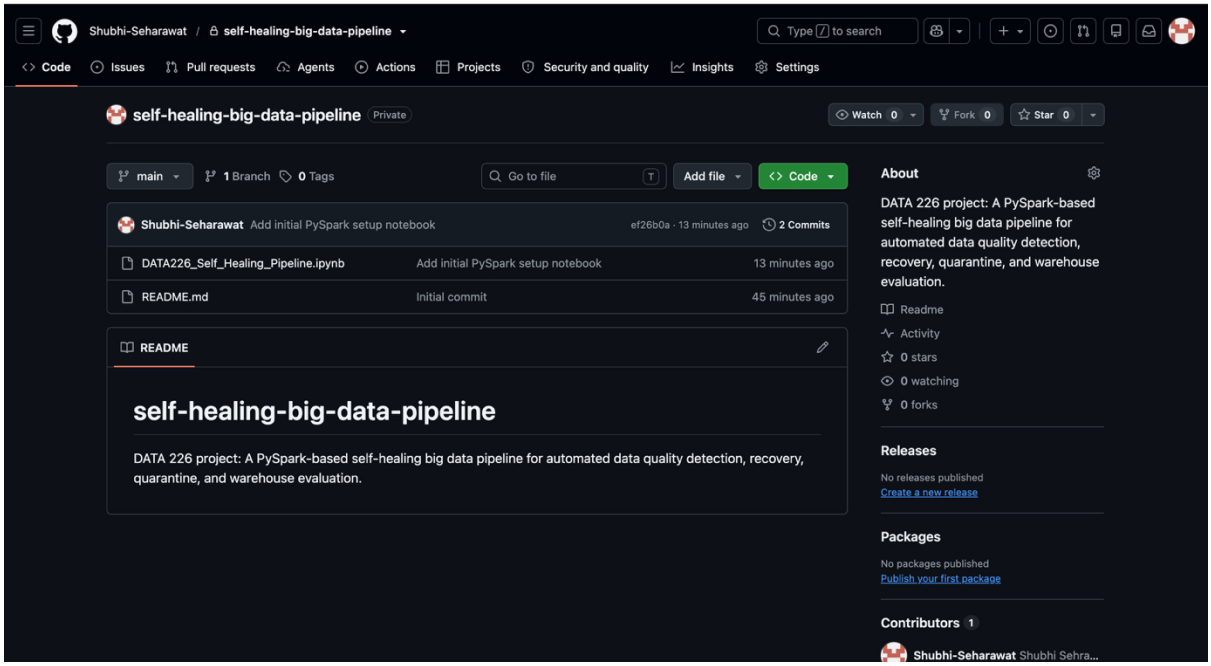
Shubhi Sehrawat (020761780)
Shaili Girishkumar Pandya (020744529)
Jyoti Priya (020092722)
Sanjana Dodda (020758985)

Project : Self-Healing Big Data Pipeline with Automatic Data Quality Recovery

This document records the technical implementation progress, environment setup, data preparation, experimental results, and challenges encountered during the development of the project.

1. Github Repository Created

Figure 1. GitHub repository showing the project README, initial PySpark notebook commit, and repository structure.

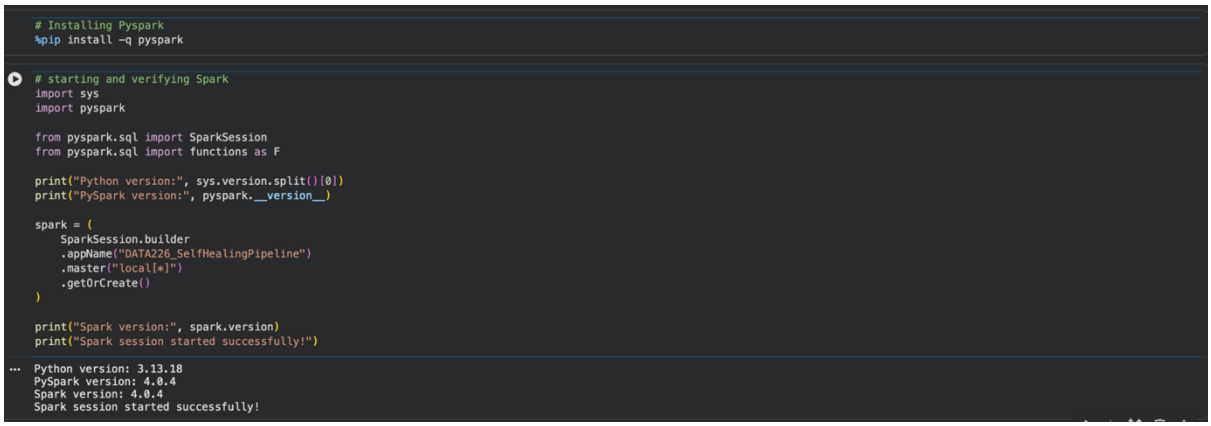


2.Initialization and Verification of the PySpark

The project environment was initialized successfully in Google Collab using PySpark. The Spark session was created in local mode, allowing the team to begin developing and testing the data processing pipeline.

The environment uses Python 3.13.18, PySpark 4.0.4, and Spark 4.0.4.

Figure 2. Successful Initialization and version verification of the PySpark environment in Google Colab.



```
# Installing Pyspark
%pip install -q pyspark

# starting and verifying Spark
import sys
import pyspark

from pyspark.sql import SparkSession
from pyspark.sql import functions as F

print("Python version:", sys.version.split()[0])
print("PySpark version:", pyspark.__version__)

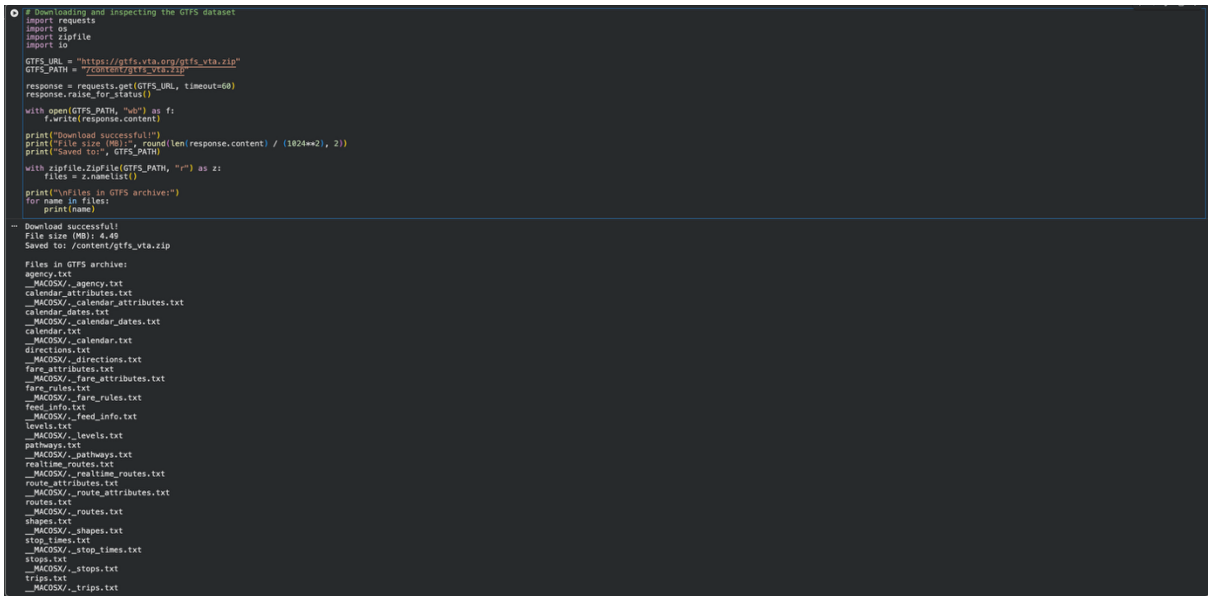
spark = (
    SparkSession.builder
    .appName("DATA226_SelfHealingPipeline")
    .master("local[*]")
    .getOrCreate()
)

print("Spark version:", spark.version)
print("Spark session started successfully!")
```

```
Python version: 3.13.18
PySpark version: 4.0.4
Spark version: 4.0.4
Spark session started successfully!
```

3. GTFS Dataset Download

Figure 3. Successful download of the VTA GTFS transit dataset, with an archive size of 4.49 MB, and archive contents.



```
# Downloading and inspecting the GTFS dataset
import requests
import os
import zipfile
import io

GTFS_URL = "https://gtfs.vta.org/gtfs_vta.zip"
GTFS_PATH = "/content/gtfs_vta.zip"

response = requests.get(GTFS_URL, timeout=60)
response.raise_for_status()

with open(GTFS_PATH, "wb") as f:
    f.write(response.content)

print("Download successful!")
print("File size (MB):", round(len(response.content) / (1024**2), 2))
print("Saved to:", GTFS_PATH)

with zipfile.ZipFile(GTFS_PATH, "r") as z:
    files = z.namelist()

print("\nFiles in GTFS archive:")
for name in files:
    print(name)
```

```
Download successful!
File size (MB): 4.49
Saved to: /content/gtfs_vta.zip

Files in GTFS archive:
agency.txt
__MACOSX/._agency.txt
calendar_attributes.txt
__MACOSX/._calendar_attributes.txt
calendar_dates.txt
__MACOSX/._calendar_dates.txt
calendar.txt
__MACOSX/._calendar.txt
directions.txt
__MACOSX/._directions.txt
fare_attributes.txt
__MACOSX/._fare_attributes.txt
fare_rules.txt
__MACOSX/._fare_rules.txt
feed_info.txt
__MACOSX/._feed_info.txt
levels.txt
__MACOSX/._levels.txt
pathways.txt
__MACOSX/._pathways.txt
realtime_routes.txt
__MACOSX/._realtime_routes.txt
route_attributes.txt
__MACOSX/._route_attributes.txt
routes.txt
__MACOSX/._routes.txt
shapes.txt
__MACOSX/._shapes.txt
stop_times.txt
__MACOSX/._stop_times.txt
stops.txt
__MACOSX/._stops.txt
trips.txt
__MACOSX/._trips.txt
```

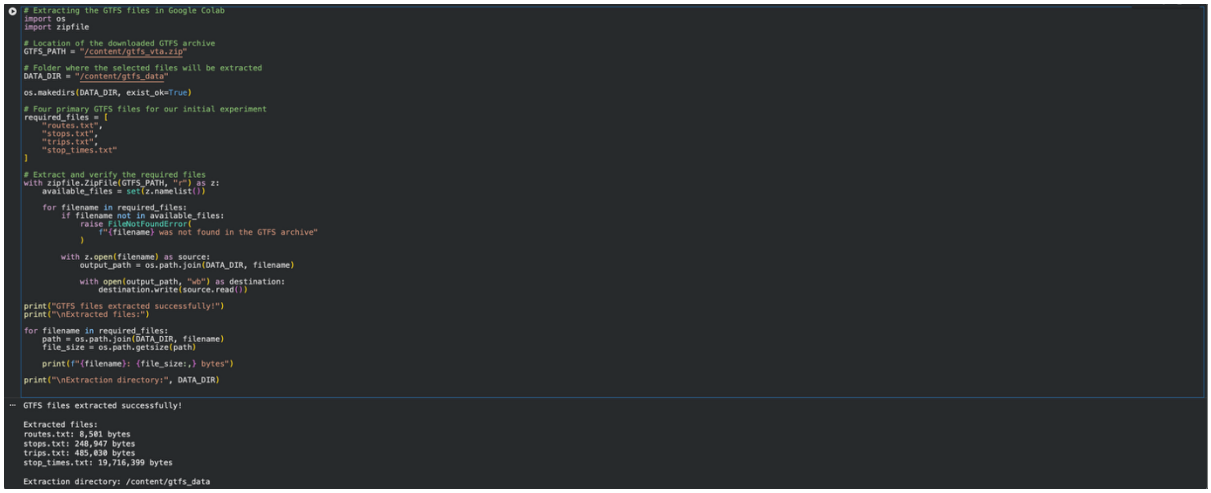
4. GTFS Data Extraction

The VTA GTFS dataset was successfully extracted in Google Colab. Four files were selected for the initial experiment: routes.txt, stops.txt, trips.txt, and stop_times.txt. These files contain transit route, stop, trip, and scheduled stop-time information. The extracted data will be used for profiling, data quality checks, and the development of the self-healing pipeline.

The extracted files have the following sizes:

- routes.txt: 8,501 bytes
- stops.txt: 248,947 bytes
- trips.txt: 485,030 bytes
- stop_times.txt: 19,716,399 bytes

Figure 4. Successful Extraction of four selected GTFS data files in Google Colab



```
# Extracting the GTFS files in Google Colab
import os
import zipfile

# Location of the downloaded GTFS archive
GTFS_PATH = "/content/gtfs_vta.zip"

# Folder where the selected files will be extracted
DATA_DIR = "/content/gtfs_data"

os.makedirs(DATA_DIR, exist_ok=True)

# Four primary GTFS files for our initial experiment
required_files = [
    "routes.txt",
    "stops.txt",
    "trips.txt",
    "stop_times.txt"
]

# Extract and verify the required files
with zipfile.ZipFile(GTFS_PATH, "r") as z:
    available_files = set(z.namelist())

    for filename in required_files:
        if filename not in available_files:
            raise FileNotFoundError(
                f"{filename} was not found in the GTFS archive"
            )

        with z.open(filename) as source:
            output_path = os.path.join(DATA_DIR, filename)

            with open(output_path, "wb") as destination:
                destination.write(source.read())

print("GTFS files extracted successfully!")
print("\nExtracted files:")

for filename in required_files:
    path = os.path.join(DATA_DIR, filename)
    file_size = os.path.getsize(path)

    print(f"{filename}: {file_size:,} bytes")

print("\nExtraction directory:", DATA_DIR)
```

```
GTFS files extracted successfully!

Extracted files:
routes.txt: 8,501 bytes
stops.txt: 248,947 bytes
trips.txt: 485,030 bytes
stop_times.txt: 19,716,399 bytes

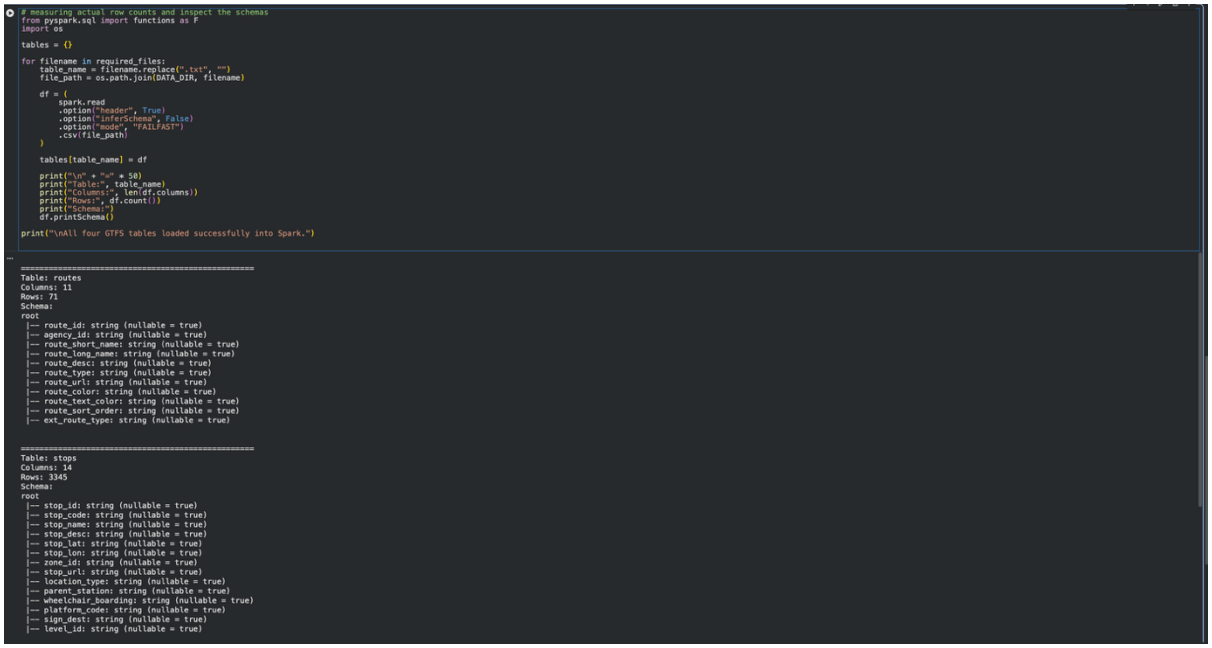
Extraction directory: /content/gtfs_data
```

5. GTFS Data Loading and Schema Inspection

The four selected GTFS files were loaded into PySpark as separate DataFrames. The routes table contains 71 rows, stops contains 3,345 rows, trips contains 8,620 rows, and stop_times contains 387,798 rows. Together, the four tables contain 399,834 rows.

The initial schema inspection showed that the tables contain information such as route identifiers, stop locations, trip identifiers, arrival times, departure times, and stop sequences. The files were initially loaded with schema inference disabled, preserving the input fields as strings for subsequent data quality checks and validation.

Figure 5(a). Row counts and schema inspection for the GTFS routes and stops tables.



```
# measuring actual row counts and inspect the schemas
from pyspark.sql import functions as F
import os

tables = {}

for filename in required_files:
    table_name = filename.replace(".txt", "")
    file_path = os.path.join(DATA_DIR, filename)

    df = (
        spark.read
        .option("header", True)
        .option("inferSchema", False)
        .option("mode", "FAILFAST")
        .csv(file_path)
    )

    tables[table_name] = df

    print("\n" + "=" * 50)
    print("Table:", table_name)
    print("Columns:", len(df.columns))
    print("Rows:", df.count())
    print("Schema:")
    df.printSchema()

print("\nAll four GTFS tables loaded successfully into Spark.")
```

```
==================================================
Table: routes
Columns: 11
Rows: 71
Schema:
root
 |-- route_id: string (nullable = true)
 |-- agency_id: string (nullable = true)
 |-- route_short_name: string (nullable = true)
 |-- route_long_name: string (nullable = true)
 |-- route_desc: string (nullable = true)
 |-- route_type: string (nullable = true)
 |-- route_url: string (nullable = true)
 |-- route_color: string (nullable = true)
 |-- route_text_color: string (nullable = true)
 |-- route_sort_order: string (nullable = true)
 |-- ext_route_type: string (nullable = true)


==================================================
Table: stops
Columns: 14
Rows: 3345
Schema:
root
 |-- stop_id: string (nullable = true)
 |-- stop_code: string (nullable = true)
 |-- stop_name: string (nullable = true)
 |-- stop_desc: string (nullable = true)
 |-- stop_lat: string (nullable = true)
 |-- stop_lon: string (nullable = true)
 |-- zone_id: string (nullable = true)
 |-- stop_url: string (nullable = true)
 |-- location_type: string (nullable = true)
 |-- parent_station: string (nullable = true)
 |-- wheelchair_boarding: string (nullable = true)
 |-- platform_code: string (nullable = true)
 |-- sign_dest: string (nullable = true)
 |-- level_id: string (nullable = true)
```

Figure 5(b). Row counts and schema inspection for the GTFS trips and stop_times tables.

```
==================================================
Table: trips
Columns: 9
Rows: 8620
Schema:
root
 |-- route_id: string (nullable = true)
 |-- service_id: string (nullable = true)
 |-- trip_id: string (nullable = true)
 |-- trip_headsign: string (nullable = true)
 |-- direction_id: string (nullable = true)
 |-- block_id: string (nullable = true)
 |-- shape_id: string (nullable = true)
 |-- wheelchair_accessible: string (nullable = true)
 |-- bikes_allowed: string (nullable = true)


==================================================
Table: stop_times
Columns: 10
Rows: 387798
Schema:
root
 |-- trip_id: string (nullable = true)
 |-- arrival_time: string (nullable = true)
 |-- departure_time: string (nullable = true)
 |-- stop_id: string (nullable = true)
 |-- stop_sequence: string (nullable = true)
 |-- stop_headsign: string (nullable = true)
 |-- pickup_type: string (nullable = true)
 |-- drop_off_type: string (nullable = true)
 |-- shape_dist_traveled: string (nullable = true)
 |-- timepoint: string (nullable = true)
```

6. Initial Data Quality Profiling

The stop_times table was analyzed to identify null and blank values across its ten columns. The profiling results showed that stop_headsign was missing or blank in all 387,798 rows, while shape_dist_traveled had 8,620 missing or blank values. The remaining eight columns had no missing or blank values according to the initial check.

These results provide a starting point for defining data quality rules. Since some GTFS fields may be optional, missing values will be evaluated against the field's intended meaning before deciding whether to repair or quarantine a record.

Figure 6. Missing and blank value counts across the ten columns of the GTFS stop_times table.



```
# Checking all 10 columns of stop_times for null or blank values.

from pyspark.sql import functions as F

# Profile missing and blank values in the stop_times table
stop_times = tables["stop_times"]

missing_exprs = [
    F.sum(
        F.when(
            F.col(c).isNull() | (F.trim(F.col(c)) == ""),
            1
        ).otherwise(0)
    ).alias(c)
    for c in stop_times.columns
]

missing_profile = stop_times.agg(*missing_exprs)

print("Missing and blank values by column in stop_times:")
display(missing_profile.toPandas())
```

```
Missing and blank values by column in stop_times:
```

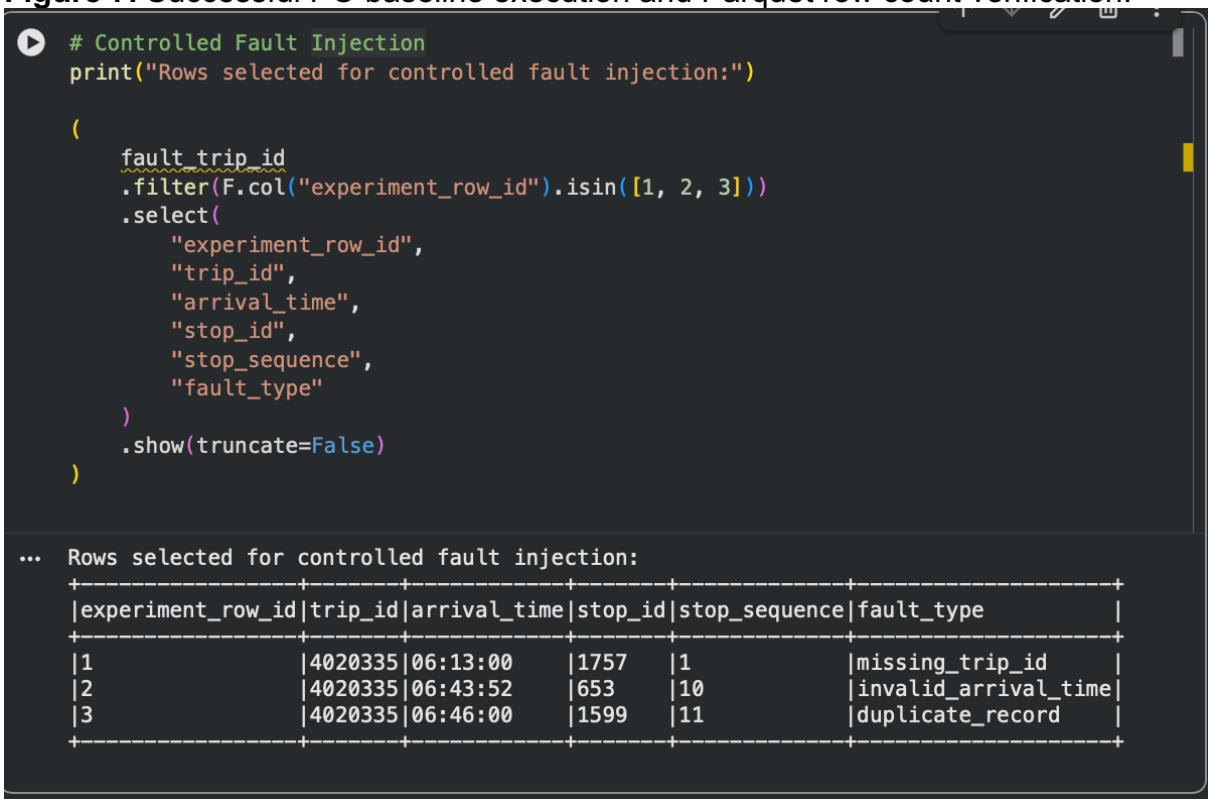
	trip_id	arrival_time	departure_time	stop_id	stop_sequence	stop_headsign	pickup_type	drop_off_type	shape_dist_traveled	timepoint
0	0	0	0	0	0	387798	0	0	8620	0

7. PO Baseline Pipeline

The P0 baseline pipeline was implemented to preserve the original stop_times dataset without applying any data repair rules. The data was written to Parquet format and then read back into Spark for verification. The source and output both contained 387,798 rows, confirming that the baseline write-and-read process preserved the row count.

This baseline will serve as the reference for evaluating the impact of fault injection, data quality detection, and recovery in subsequent experiments.

Figure 7. Successful PO baseline execution and Parquet row count verification.



```
# Controlled Fault Injection
print("Rows selected for controlled fault injection:")

(
    fault_trip_id
    .filter(F.col("experiment_row_id").isin([1, 2, 3]))
    .select(
        "experiment_row_id",
        "trip_id",
        "arrival_time",
        "stop_id",
        "stop_sequence",
        "fault_type"
    )
    .show(truncate=False)
)
```

```
Rows selected for controlled fault injection:
+-----------------+-------+------------+-------+-------------+--------------------+
|experiment_row_id|trip_id|arrival_time|stop_id|stop_sequence|fault_type          |
+-----------------+-------+------------+-------+-------------+--------------------+
|1                |4020335|06:13:00    |1757   |1            |missing_trip_id     |
|2                |4020335|06:43:52    |653    |10           |invalid_arrival_time|
|3                |4020335|06:46:00    |1599   |11           |duplicate_record    |
+-----------------+-------+------------+-------+-------------+--------------------+
```

8. Controlled Fault Injection Preparation

Three source records were selected from the stop_times dataset to prepare a controlled fault-injection experiment. The planned fault types are a missing trip identifier, an invalid arrival time, and a duplicated record. The selected records and their intended fault types will be tracked to establish ground truth for evaluating data quality detection.

At this stage, the records have only been selected and labelled. The faults have not yet been injected.

Figure 8. Selection of source records and their intended fault types.

```
# Building the P0 baseline pipeline

import os

BASELINE_DIR = "/content/pipeline_outputs/p0_baseline"

os.makedirs(os.path.dirname(BASELINE_DIR), exist_ok=True)

# P0: preserve the original data without repair rules
(
    tables["stop_times"]
    .write
    .mode("overwrite")
    .parquet(BASELINE_DIR)
)

# Read the saved output back to verify it
baseline_df = spark.read.parquet(BASELINE_DIR)

source_count = tables["stop_times"].count()
baseline_count = baseline_df.count()

print("P0 baseline pipeline completed.")
print("Source rows:", source_count)
print("Parquet rows:", baseline_count)
print("Row counts match:", source_count == baseline_count)
print("Output directory:", BASELINE_DIR)
```

```
P0 baseline pipeline completed.
Source rows: 387798
Parquet rows: 387798
Row counts match: True
Output directory: /content/pipeline_outputs/p0_baseline
```

9. Controlled Fault Injection

Three controlled data quality faults were introduced into a separate copy of the stop_times dataset. The first fault replaced a trip identifier with a null value, the second replaced an arrival time with the invalid value "invalid_time", and the third duplicated a selected record.

The original dataset contained 387,798 rows, while the fault-injected dataset contained 387,799 rows. This confirms that the additional duplicate record was appended. The original source data and P0 baseline were preserved for comparison.

Figure 9. Verification of the injected missing identifier , invalid arrival time and duplicate record.

```
# creating a seperate fault injection dataframe
from pyspark.sql import functions as F

# Create a separate dataset for the fault-injection experiment.
# The original tables["stop_times"] and P0 baseline remain unchanged.

faulty_base = indexed_df

# Inject fault 1: missing trip_id in row 1
faulty_base = faulty_base.withColumn(
    "trip_id",
    F.when(
        F.col("experiment_row_id") == 1,
        F.lit(None).cast("string")
    ).otherwise(F.col("trip_id"))
)

# Inject fault 2: invalid arrival_time in row 2
faulty_base = faulty_base.withColumn(
    "arrival_time",
    F.when(
        F.col("experiment_row_id") == 2,
        F.lit("invalid_time")
    ).otherwise(F.col("arrival_time"))
)

# Inject fault 3: duplicate row 3
duplicate_row = (
    faulty_base
    .filter(F.col("experiment_row_id") == 3)
    .withColumn("experiment_row_id", F.lit(387799))
)

faulty_data = faulty_base.unionByName(duplicate_row)

print("Fault injection completed.")
print("Original row count:", indexed_df.count())
print("Fault-injected row count:", faulty_data.count())

print("\nInjected fault examples:")
faulty_data.filter(
    F.col("experiment_row_id").isin([1, 2, 3, 387799])
).select(
    "experiment_row_id",
    "trip_id",
    "arrival_time",
    "stop_id",
    "stop_sequence"
).show(truncate=False)
```

```
Fault injection completed.
Original row count: 387798
Fault-injected row count: 387799

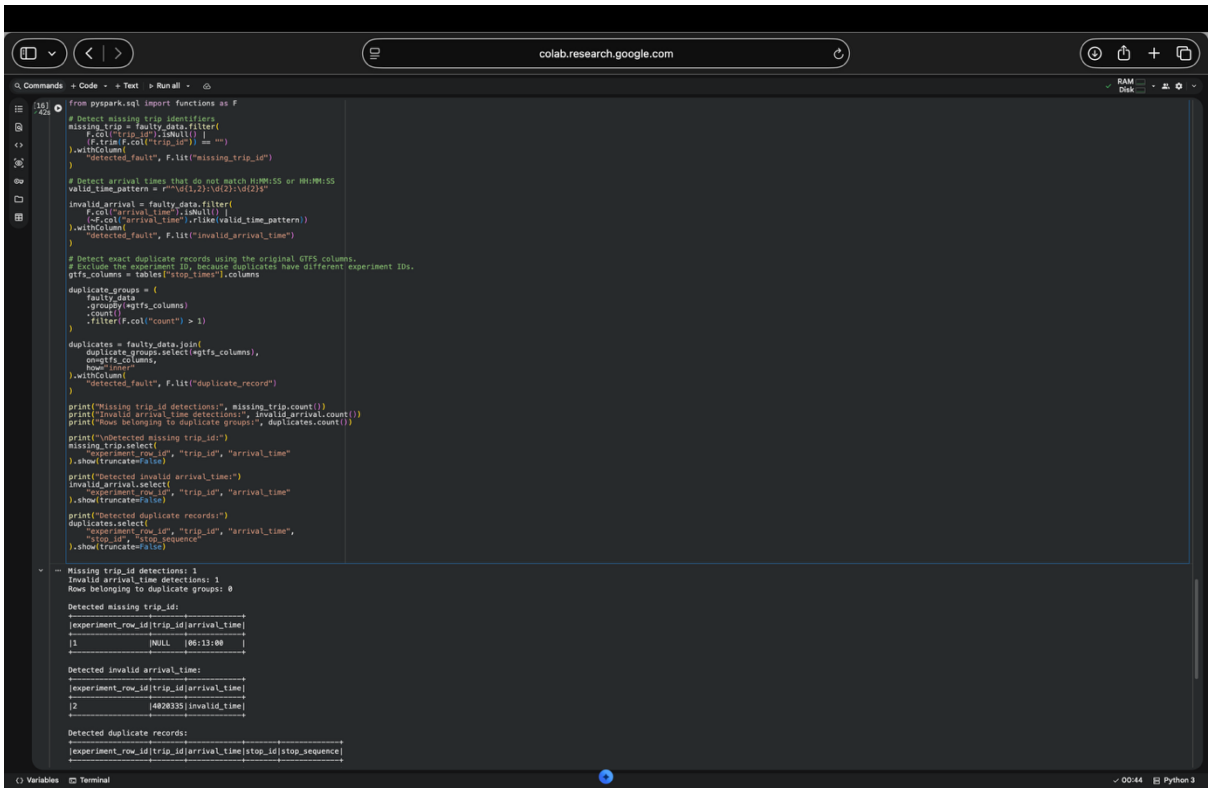
Injected fault examples:
+----------------+-------+------------+-------+-------------+
|experiment_row_id|trip_id|arrival_time|stop_id|stop_sequence|
+----------------+-------+------------+-------+-------------+
|1               |NULL   |06:13:00    |1757   |1            |
|2               |4020335|invalid_time|653    |10           |
|3               |4020335|06:46:00    |1599   |11           |
|387799          |4020335|06:46:00    |1599   |11           |
+----------------+-------+------------+-------+-------------+
```

10.1 Initial Fault Detection Attempt

The first detection attempt successfully identified one record with a missing trip identifier and one record with an invalid arrival time. However, the join-based duplicate detection approach returned zero matching rows, even though a duplicate had been deliberately introduced into the dataset.

This result showed that the duplicate detection logic needed further investigation and improvement.

Figure 10. Initial Fault detection results showing successful detection of the missing identifier and invalid arrival time but no duplicate records detected.



10.2. Correction of Duplicate Detection

The initial duplicate detection issue was addressed by replacing the join-based approach with a Spark window function. The updated implementation grouped records using all original GTFS columns and counted the occurrences of each record.

The corrected detector identified two rows belonging to the same duplicate group. Both rows had a duplicate count of two, confirming that the deliberately injected duplicate was detected successfully.

Figure 11. Corrected duplicate detection using a Spark window function identifying both copies of the duplicated record.



```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

# Use the original GTFS columns, excluding experiment_row_id
gtfs_columns = tables["stop_times"].columns

# Count identical GTFS records, including rows with null fields
duplicate_window = Window.partitionBy(*gtfs_columns)

duplicate_check = faulty_data.withColumn(
    "duplicate_count",
    F.count("*").over(duplicate_window)
)

duplicates = (
    duplicate_check
    .filter(F.col("duplicate_count") > 1)
    .withColumn(
        "detected_fault",
        F.lit("duplicate_record")
    )
)

print("Rows belonging to duplicate groups:", duplicates.count())

duplicates.select(
    "experiment_row_id",
    "trip_id",
    "arrival_time",
    "stop_id",
    "stop_sequence",
    "duplicate_count"
).show(truncate=False)
```

```
Rows belonging to duplicate groups: 2
+-----------------+-------+------------+-------+-------------+---------------+
|experiment_row_id|trip_id|arrival_time|stop_id|stop_sequence|duplicate_count|
+-----------------+-------+------------+-------+-------------+---------------+
|3                |4020335|06:46:00    |1599   |11           |2              |
|387799           |4020335|06:46:00    |1599   |11           |2              |
+-----------------+-------+------------+-------+-------------+---------------+
```

10.3. Lesson Learned

This experiment showed why it is important to test data quality rules using examples with known errors. In the first attempt, the code failed to detect the duplicate records. After changing the approach, it successfully identified both copies. This helped us find the problem, fix the code, and improve the accuracy of duplicate detection.

11. Data Recovery and Quarantine

The recovery stage separated invalid records from the data that could be retained. One record with a missing trip identifier and one record with an invalid arrival time were moved to quarantine. The duplicated record was handled by retaining one copy and removing the extra copy.

The fault-injected dataset contained 387,799 rows. After quarantining two invalid records and removing one duplicate, the recovered dataset contained 387,796 rows. These results matched the expected outcome of the controlled experiment.

Figure 12. Recovery results showing quarantined records and the recovered dataset row count .

```python
# data recovery and quarantine

from pyspark.sql import functions as F
from pyspark.sql.window import Window

# Original GTFS columns (excluding experiment_row_id)
gtfs_columns = tables["stop_times"].columns

# Rule 1: Identify records with missing trip IDs
missing_trip_condition = (
    F.col("trip_id").isNull() |
    (F.trim(F.col("trip_id")) == "")
)

# Rule 2: Identify invalid arrival times
valid_time_pattern = r"^\d{1,2}:\d{2}:\d{2}$"

invalid_time_condition = (
    F.col("arrival_time").isNull() |
    (~F.col("arrival_time").rlike(valid_time_pattern))
)

# Quarantine records that fail either validation rule
quarantine_df = faulty_data.filter(
    missing_trip_condition | invalid_time_condition
)

# Keep records that pass both validation rules
valid_df = faulty_data.filter(
    ~missing_trip_condition & ~invalid_time_condition
)

# Rule 3: Remove exact duplicate GTFS records, keeping one copy
dedup_window = Window.partitionBy(*gtfs_columns).orderBy(
    F.col("experiment_row_id")
)

recovered_df = (
    valid_df
    .withColumn(
        "dedup_rank",
        F.row_number().over(dedup_window)
    )
    .filter(F.col("dedup_rank") == 1)
    .drop("dedup_rank", "experiment_row_id")
)

# Remove experiment-only IDs from quarantine output too
quarantine_output = quarantine_df.drop("experiment_row_id")

print("Recovery stage completed.")
print("Original source rows:", tables["stop_times"].count())
print("Fault-injected rows:", faulty_data.count())
print("Quarantined rows:", quarantine_output.count())
print("Recovered rows:", recovered_df.count())

print("\nQuarantined records:")
quarantine_output.select(
    "trip_id", "arrival_time", "stop_id", "stop_sequence"
).show(truncate=False)
```

```
Recovery stage completed.
Original source rows: 387798
Fault-injected rows: 387799
Quarantined rows: 2
Recovered rows: 387796

Quarantined records:
+-------+------------+-------+-------------+
|trip_id|arrival_time|stop_id|stop_sequence|
+-------+------------+-------+-------------+
|NULL   |06:13:00    |1757   |1            |
|4020335|invalid_time|653    |10           |
+-------+------------+-------+-------------+
```

12. Initial Evaluation Results

The detection system was evaluated using three known fault events that were deliberately introduced into the dataset. These included a missing trip identifier, an invalid arrival time, and a duplicated record. The detection rules identified all three injected fault types.

In this controlled experiment, detection precision and recall were both 100%. These results indicate that the implemented rules correctly identified the faults introduced during testing. However, the experiment included only three known fault events, so the results should not be treated as a measure of performance on all possible data quality problems.

Figure 13. Initial detection evaluation showing 100% precision and 100% recall in the controlled experiment.

```python
# Evaluating detection performance

# Ground truth: three deliberately injected fault events
true_faults = 3

# Detection results verified in our previous cells:
detected_missing_trip = missing_trip.count()
detected_invalid_time = invalid_arrival.count()

# A duplicate event is one duplicated group, not two affected rows.
detected_duplicate_groups = (
    duplicate_check
    .filter(F.col("duplicate_count") > 1)
    .select(*gtfs_columns)
    .distinct()
    .count()
)

true_positives = (
    min(detected_missing_trip, 1)
    + min(detected_invalid_time, 1)
    + min(detected_duplicate_groups, 1)
)

precision = true_positives / true_faults
recall = true_positives / true_faults

print("Known injected fault events:", true_faults)
print("Missing trip_id detections:", detected_missing_trip)
print("Invalid arrival_time detections:", detected_invalid_time)
print("Duplicate groups detected:", detected_duplicate_groups)
print("True positive fault types:", true_positives)
print(f"Detection precision (controlled test): {precision:.2%}")
print(f"Detection recall (controlled test): {recall:.2%}")
```

```
Known injected fault events: 3
Missing trip_id detections: 1
Invalid arrival_time detections: 1
Duplicate groups detected: 1
True positive fault types: 3
Detection precision (controlled test): 100.00%
Detection recall (controlled test): 100.00%
```

13. Saving Recovery Outputs

The recovered dataset and quarantined records were saved separately in Parquet format. The recovered dataset contains 387,796 rows, while the quarantine output contains two records that failed validation. Both outputs were read back into Spark to verify their row counts after saving.

Separating these outputs allows valid records to continue through the pipeline while keeping invalid records available for further inspection.

Figure 14. Recovered dataset and quarantine outputs saved and verified in Parquet format

```python
# Saving the recovered data and quarantine records

import os

OUTPUT_DIR = "/content/pipeline_outputs"

RECOVERED_PATH = os.path.join(
    OUTPUT_DIR, "p2_recovered"
)

QUARANTINE_PATH = os.path.join(
    OUTPUT_DIR, "p2_quarantine"
)

# Save the recovered dataset
recovered_df.write.mode("overwrite").parquet(
    RECOVERED_PATH
)

# Save the quarantined records
quarantine_output.write.mode("overwrite").parquet(
    QUARANTINE_PATH
)

# Read back and verify the saved outputs
saved_recovered = spark.read.parquet(RECOVERED_PATH)
saved_quarantine = spark.read.parquet(QUARANTINE_PATH)

print("Recovery outputs saved successfully.")
print("Saved recovered rows:", saved_recovered.count())
print("Saved quarantine rows:", saved_quarantine.count())
print("Recovered output:", RECOVERED_PATH)
print("Quarantine output:", QUARANTINE_PATH)
```

```
Recovery outputs saved successfully.
Saved recovered rows: 387796
Saved quarantine rows: 2
Recovered output: /content/pipeline_outputs/p2_recovered
Quarantine output: /content/pipeline_outputs/p2_quarantine
```